

0.1. Parte B

a) Intervalos de confianza Bootstrap para a y b .

Fundamento analítico: Según el método Bootstrap no paramétrico, asumimos que la Función de Distribución Empírica (FDE) de los datos observados es la mejor aproximación a la distribución poblacional. Generamos B muestras bootstrap (con $B \geq 1000$ para estabilizar los cuantiles) extrayendo n pares de datos (x_i, y_i) con *reemplazo*. Para cada muestra, calculamos los estimadores de mínimos cuadrados ordinarios (OLS) $\hat{a}^*$ y $\hat{b}^*$. El intervalo de confianza del 95% se construye mediante el *método percentil*, tomando los valores en las posiciones $0,025 \times B$ y $0,975 \times B$ de las distribuciones ordenadas de $\hat{a}^*$ y $\hat{b}^*$.

Comparación: Los intervalos Bootstrap son puramente frecuentistas y dependen exclusivamente de los datos, sin incorporar la previa. Dado que la previa usada en la Parte A era amplia ($\sigma^2 = 100$), los intervalos de credibilidad bayesianos y los de confianza Bootstrap deberían ser numéricamente muy similares, aunque su interpretación filosófica difiere (probabilidad del parámetro vs. cobertura a largo plazo del procedimiento).

```
1 B = 2000 # Numero de replicas bootstrap
2 a_boot, b_boot = [], []
3 for _ in range(B):
4     idx = np.random.choice(n, size=n, replace=True)
5     X_b = np.column_stack([np.ones(n), x[idx]])
6     y_b = y[idx]
7     theta_b = np.linalg.lstsq(X_b, y_b, rcond=None)[0]
8     a_boot.append(theta_b[0]); b_boot.append(theta_b[1])
9
10 ci_a_boot = np.percentile(a_boot, [2.5, 97.5])
11 ci_b_boot = np.percentile(b_boot, [2.5, 97.5])
12 print(f"IC Bootstrap 95% a: {ci_a_boot}, b: {ci_b_boot}")
```

b) Chequeo predictivo posterior y prueba de Kolmogorov-Smirnov.

Fundamento analítico: Para validar el modelo, comparamos la distribución de los residuos observados con la de residuos simulados a partir del modelo predictivo posterior. Para cada una de las 10^4 muestras $(\tilde{a}, \tilde{b})$, generamos datos sintéticos $\tilde{y}_i = \tilde{a} + \tilde{b}x_i + \varepsilon_i$, con $\varepsilon_i \sim \mathcal{N}(0, 0.3^2)$. Calculamos los residuos sintéticos $\tilde{r}_i = \tilde{y}_i - (\tilde{a} + \tilde{b}x_i)$ (que por construcción siguen $\mathcal{N}(0, 0.3^2)$) y los residuos observados $r_i = y_i - (\hat{a}_{MAP} + \hat{b}_{MAP}x_i)$. Aplicamos la prueba K-S de dos muestras, que evalúa la máxima diferencia absoluta $D = \max_x |S_{N_1}(x) - S_{N_2}(x)|$ entre las funciones de distribución acumulada empíricas de ambos conjuntos de residuos. Un valor p alto indica que no hay evidencia para rechazar la hipótesis nula de que ambos conjuntos de residuos provienen de la misma distribución, validando la suposición de normalidad y homocedasticidad del modelo.

```
1 from scipy import stats
2
3 # Residuos observados usando estimadores MAP
4 y_pred_map = a_map + b_map * x
5 res_obs = y - y_pred_map
6
7 # Residuos sinteticos (simulamos 10^4 conjuntos, tomamos uno representativo o
8   aplanamos)
9 # Para la prueba K-S, aplanamos las simulaciones para tener una gran muestra
10  de referencia
11 res_sim = np.random.normal(0, sigma, size=(10000, n)).flatten()
12
13 # Prueba de Kolmogorov-Smirnov de dos muestras
14 D_stat, p_value = stats.ks_2samp(res_obs, res_sim)
15 print(f"Estadístico K-S D = {D_stat:.4f}, p-valor = {p_value:.4f}")
```

c) Modelo cuadrático y comparación de poder predictivo con Bootstrap.

Fundamento analítico: El modelo alternativo es $M_i = a + bx_i + cx_i^2 + \varepsilon_i$. Para encontrar los valores MAP, extendemos el sistema lineal de la Parte A incluyendo el parámetro c y su término de regularización de la previa ($\sigma^2/100$ en la diagonal de la matriz de precisión).

Para comparar el poder predictivo, utilizamos Bootstrap para estimar la distribución del Error Cuadrático Medio (MSE) de ambos modelos. El modelo con la mediana del MSE bootstrap más baja tiene mejor ajuste, pero debemos aplicar el principio de parsimonia (navaja de Occam): si la mejora en el MSE es marginal, el modelo lineal es preferible por tener menos parámetros y menor riesgo de sobreajuste.

```
1 # 1. Estimadores MAP del modelo cuadrático
```

```

2 x2 = x**2
3 sum_x3 = (x**3).sum(); sum_x4 = (x**4).sum(); sum_x2y = (x2*y).sum()
4 M_quad = np.array([
5     [n + sigma**2/100, sum_x, sum_x2],
6     [sum_x, sum_x2 + sigma**2/100, sum_x3],
7     [sum_x2, sum_x3, sum_x4 + sigma**2/100]
8 ])
9 Z_quad = np.array([sum_y, sum_xy, sum_x2y])
10 a_q, b_q, c_q = np.linalg.solve(M_quad, Z_quad)
11
12 # 2. Comparacion de poder predictivo via Bootstrap (MSE)
13 mse_lin, mse_quad = [], []
14 for _ in range(B):
15     idx = np.random.choice(n, size=n, replace=True)
16     x_b, y_b = x[idx], y[idx]
17
18     # MSE Lineal
19     X_b_lin = np.column_stack([np.ones(n), x_b])
20     theta_lin = np.linalg.lstsq(X_b_lin, y_b, rcond=None)[0]
21     mse_lin.append(np.mean((y_b - X_b_lin @ theta_lin)**2))
22
23     # MSE Cuadratico
24     X_b_quad = np.column_stack([np.ones(n), x_b, x_b**2])
25     theta_quad = np.linalg.lstsq(X_b_quad, y_b, rcond=None)[0]
26     mse_quad.append(np.mean((y_b - X_b_quad @ theta_quad)**2))
27
28 print(f"MSE Lineal (mediana bootstrap): {np.median(mse_lin):.4f}")
29 print(f"MSE Cuadratico (mediana bootstrap): {np.median(mse_quad):.4f}")

```

Nota: Si la diferencia entre las medianas del MSE es pequeña, el modelo lineal de la Parte A es estadísticamente suficiente, evitando la complejidad innecesaria del término cuadrático.